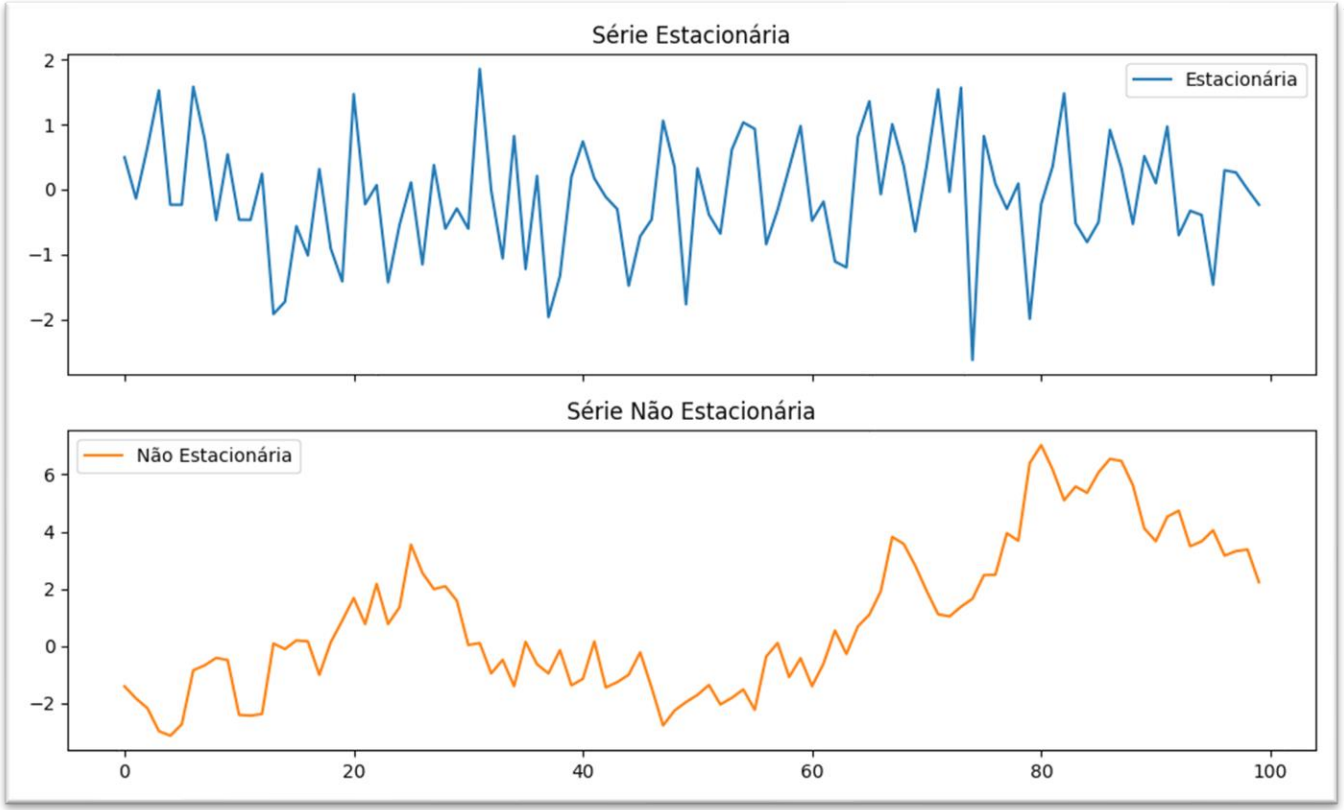
Diferenciação

Daniel de Freitas

Instituto J&F
Centro de Educação para os Negócios

Diferenciação: definição

- Muitas séries temporais não são estacionárias: apresentam tendência ou sazonalidade ao longo do tempo.
- Os modelos da família AR, MA e ARMA exigem estacionariedade.
- A **diferenciação** é a técnica matemática usada para **remover tendência ou sazonalidade, tornando a série estacionária**.



Tipos de Diferenciação

Tipos de diferenciação:

Existe vários tipos de diferenciações para casos diversos:

- Primeira ordem;
- Ordem superiores;
- Sazonal
- Combinação

As duas primeiras funcionam melhor para quando não há sazonalidade. Vamos entender cada uma delas.

Diferenciação Simples

Diferenciação Simples:

É a forma mais comum. Consiste em **subtrair o valor atual da série pelo valor anterior. Serve para eliminar tendência linear.**

$$\nabla y_t = y_t - y_{t-1}$$

y_t: valor da série no tempo t,

y_{t-1}: valor da série no tempo anterior,

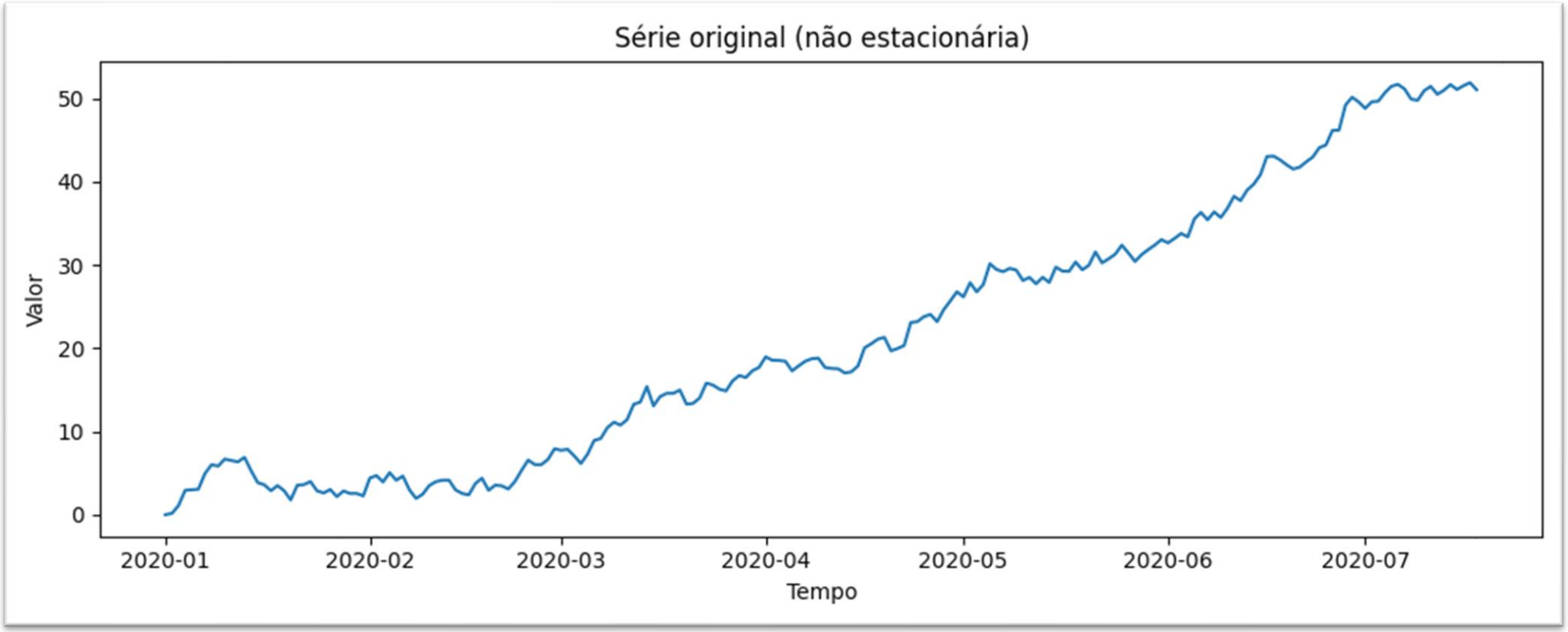
∇y_t: primeira diferença.

Implementação: gerando

- Vamos gerar uma série não estacionária e visualiza-la para aplicarmos a diferenciação

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller, kpss

# reprodutibilidade
np.random.seed(42)

# serie NÃO estacionária
n = 200
eps = np.random.normal(0, 1, n)
drift = 0.3
y = np.zeros(n)
for t in range(1, n):
    y[t] = y[t-1] + drift + eps[t]

idx = pd.date_range(start="2020-01-01", periods=n, freq="D")
serie = pd.Series(y, index=idx, name="nao estacionaria")

# serie originall
plt.figure(figsize=(10,4))
plt.plot(serie.index, serie.values)
plt.title("Série original (não estacionária)")
plt.xlabel("Tempo")
plt.ylabel("Valor")
plt.tight_layout()
plt.show()
```

Implementação: gerando



Testando estacionariedade: criando funções

```python
# suprimir todos os warnings
warnings.filterwarnings("ignore")


# funcoes de teste
def teste_adf(x):
    res = adfuller(x)
    resultados = {
        "Estatística ADF": res[0],
        "p-valor": res[1],
        "Críticos 1/5/10%": res[4],
    }
    print('ADF')
    for k, v in resultados.items():
        print(f"  {k}: {v}")
    print()



def teste_kpss_(x):
    res = kpss(x)
    resultados = {
        "Estatística KPSS": res[0],
        "p-valor": res[1],
        "Críticos 10/5/2.5/1%": res[3],
    }
    print('KPSS')
    for k, v in resultados.items():
        print(f"  {k}: {v}")
    print()
```

Testando estacionariedade:

Testando estacionariedade - antes da diferenciação

```python
# testes na série original
adf_orig = teste_adf(serie)
kpss_orig = teste_kpss_(serie)
```

[24] ✓ 0.0s

```
ADF
  Estatística ADF: 0.5916531122925627
  p-valor: 0.9874080755717005
  Críticos 1/5/10%: {'1%': -3.4636447617687436, '5%': -2.8761761179270766, '10%': -2.57457158581854}

KPSS
  Estatística KPSS: 2.029069889393346
  p-valor: 0.01
  Críticos 10/5/2.5/1%: {'10%': 0.347, '5%': 0.463, '2.5%': 0.574, '1%': 0.739}
```

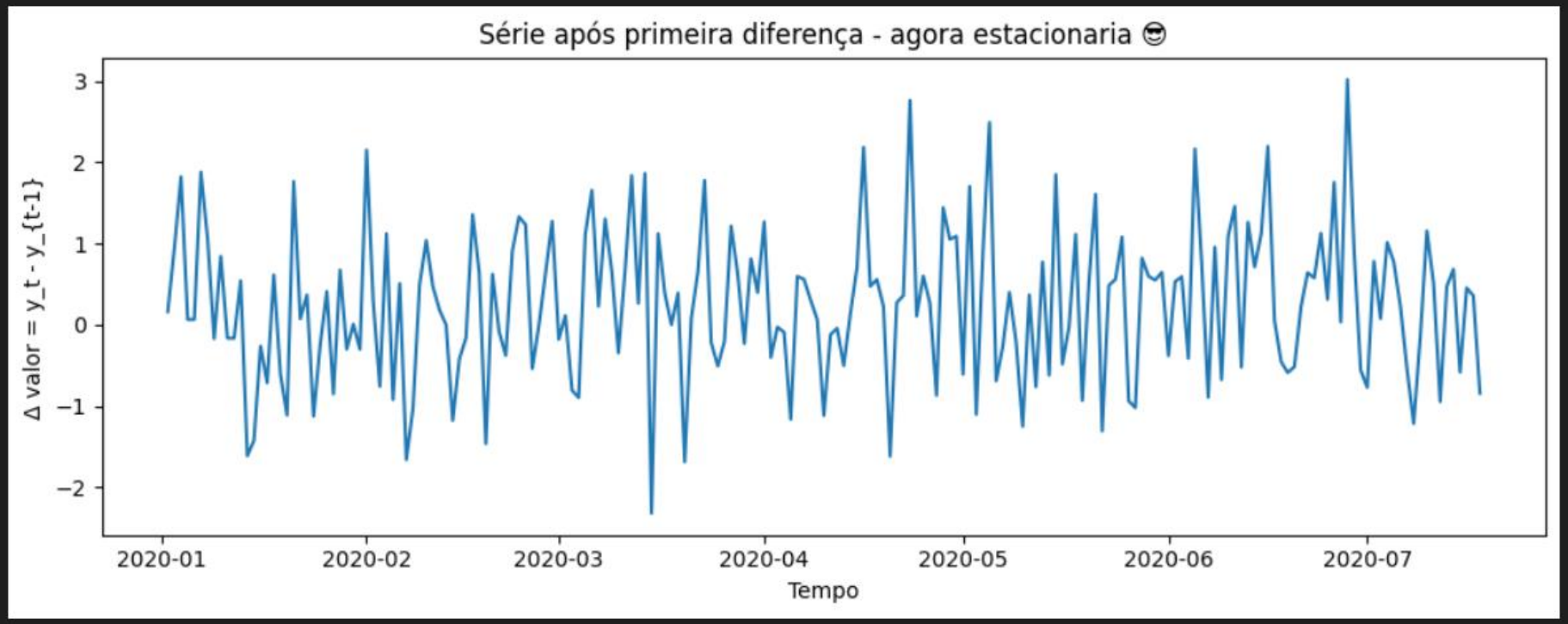
Realizando a diferenciação:

Diferenciação Simples

```
# diferenciação de primeira ordem
diff1 = serie.diff(1).dropna()

# testes na série diferenciada
adf_diff = teste_adf(diff1)
kpss_diff = teste_kpss_(diff1)

plt.figure(figsize=(10,4))
plt.plot(diff1.index, diff1.values)
plt.title("Série após primeira diferença - agora estacionaria 😎")
plt.xlabel("Tempo")
plt.ylabel("Δ valor = y_t - y_{t-1}")
plt.tight_layout()
plt.show()
```

Realizando a diferenciação:

```
ADF
  Estatística ADF: -14.69177492414318
  p-valor: 3.0415339027296236e-27
  Críticos 1/5/10%: {'1%': -3.4638151713286316, '5%': -2.876250632135043, '10%': -2.574611347821651}

KPSS
  Estatística KPSS: 0.20147151871110763
  p-valor: 0.1
  Críticos 10/5/2.5/1%: {'10%': 0.347, '5%': 0.463, '2.5%': 0.574, '1%': 0.739}
```



Retornando a série original:

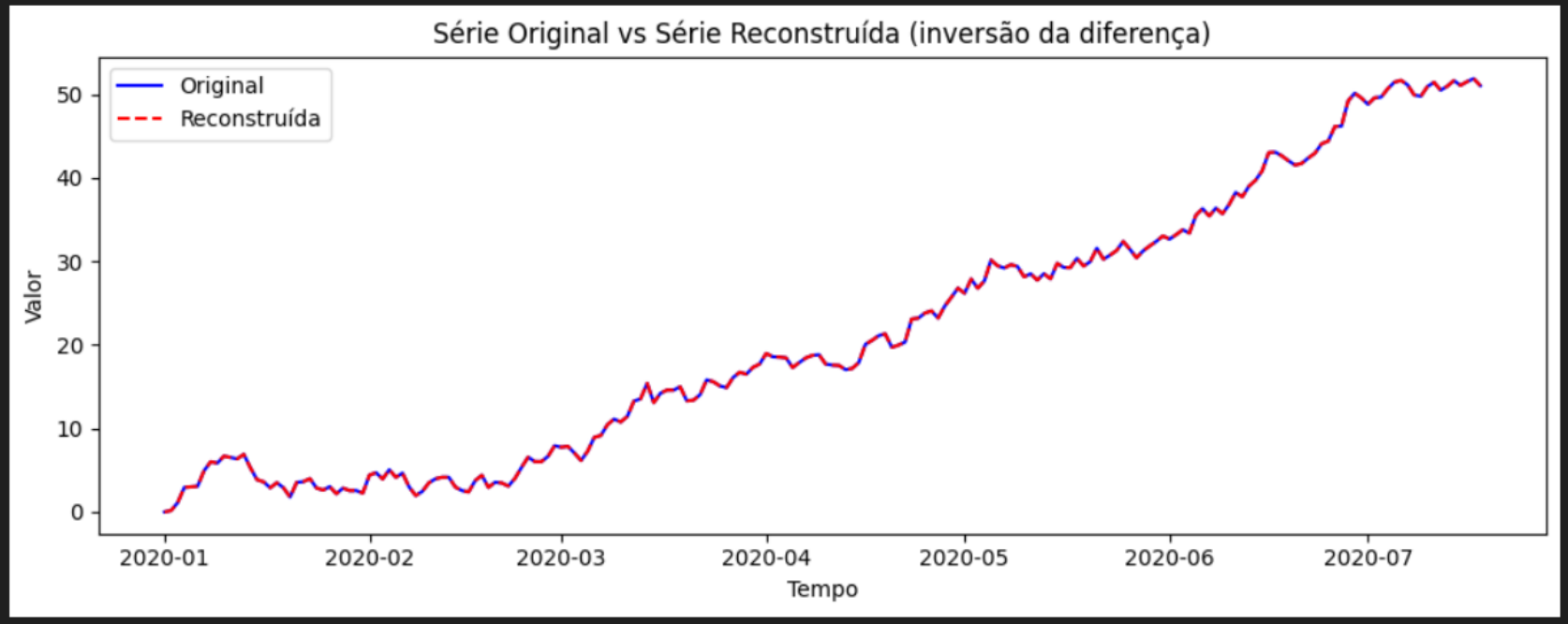
🚨 🚨 **Ao final da diferenciação devemos retornar a série original, principalmente se quisermos fazer uma previsão !!** 🚨 🚨

Inversão - voltando a série ao original

```python
# inversao da diferenciação para reconstruir a série original
y0 = serie.iloc[0]  # primeiro valor original (ponto âncora)
reconstruida = pd.Series([y0], index=[serie.index[0]], name="Reconstruída")
reconstruida = pd.concat([reconstruida, (y0 + diff1.cumsum())])

# comparacao numérica
comparacao = pd.DataFrame({
    "Original": serie,
    "Reconstruída": reconstruida
})
print("\nComparação: série original vs reconstruída da diferença")
print(comparacao.head(10))  # mostranndo os 10 primeiros valores, apennas para comparar

# plotagem sobreposta
plt.figure(figsize=(10,4))
plt.plot(serie.index, serie.values, label="Original", color="blue")
plt.plot(reconstruida.index, reconstruida.values, label="Reconstruída", color="red", linestyle="--")
plt.title("Série Original vs Série Reconstruída (inversão da diferença)")
plt.xlabel("Tempo")
plt.ylabel("Valor")
plt.legend()
plt.tight_layout()
plt.show()
```

Retornando a série original:

```
Comparação: série original vs reconstruída da diferença
            Original  Reconstruída
2020-01-01  0.000000      0.000000
2020-01-02  0.161736      0.161736
2020-01-03  1.109424      1.109424
2020-01-04  2.932454      2.932454
2020-01-05  2.998301      2.998301
2020-01-06  3.064164      3.064164
2020-01-07  4.943377      4.943377
2020-01-08  6.010811      6.010811
2020-01-09  5.841337      5.841337
2020-01-10  6.683897      6.683897
```



Diferenciação de Ordem Superior

Diferenciação de ordem superior:

Se a série ainda não for estacionária após a primeira diferença, podemos aplicar novamente:

$$\nabla^2 y_t = \nabla(\nabla y_t) = (y_t - y_{t-1}) - (y_{t-1} - y_{t-2}) = y_t - 2y_{t-1} + y_{t-2}$$

Diferenciação de ordem d, ou seja, aplicar d vezes a diferenciação:

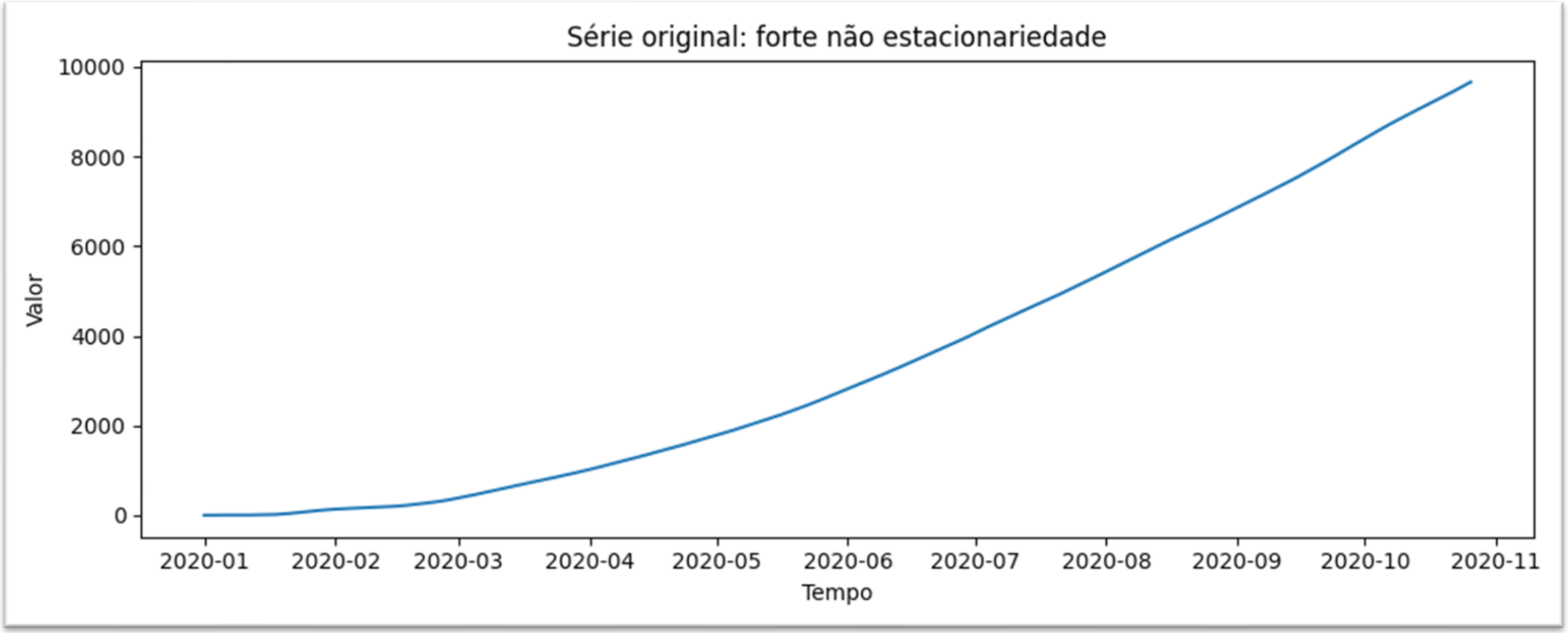
$$\nabla^d y_t = (1 - B)^d y_t$$

Implementação: gerando

- Vamos gerar uma série não estacionária e visualiza-la para aplicarmos a diferenciação de segunda ordem

```
# reprodutibilidade
np.random.seed(123)

# construindo serie de segunda ordem para testarmos
n = 300
drift = 0.2
v = np.random.normal(0, 1, n)
s = np.zeros(n)
y = np.zeros(n)

for t in range(1, n):
    s[t] = s[t-1] + drift + v[t]
for t in range(1, n):
    y[t] = y[t-1] + s[t]

idx = pd.date_range(start="2020-01-01", periods=n, freq="D")
serie = pd.Series(y, index=idx, name="y")

plt.figure(figsize=(10,4))
plt.plot(serie.index, serie.values)
plt.title("Série original: forte não estacionariedade")
plt.xlabel("Tempo")
plt.ylabel("Valor")
plt.tight_layout()
plt.show()
```

Implementação: gerando

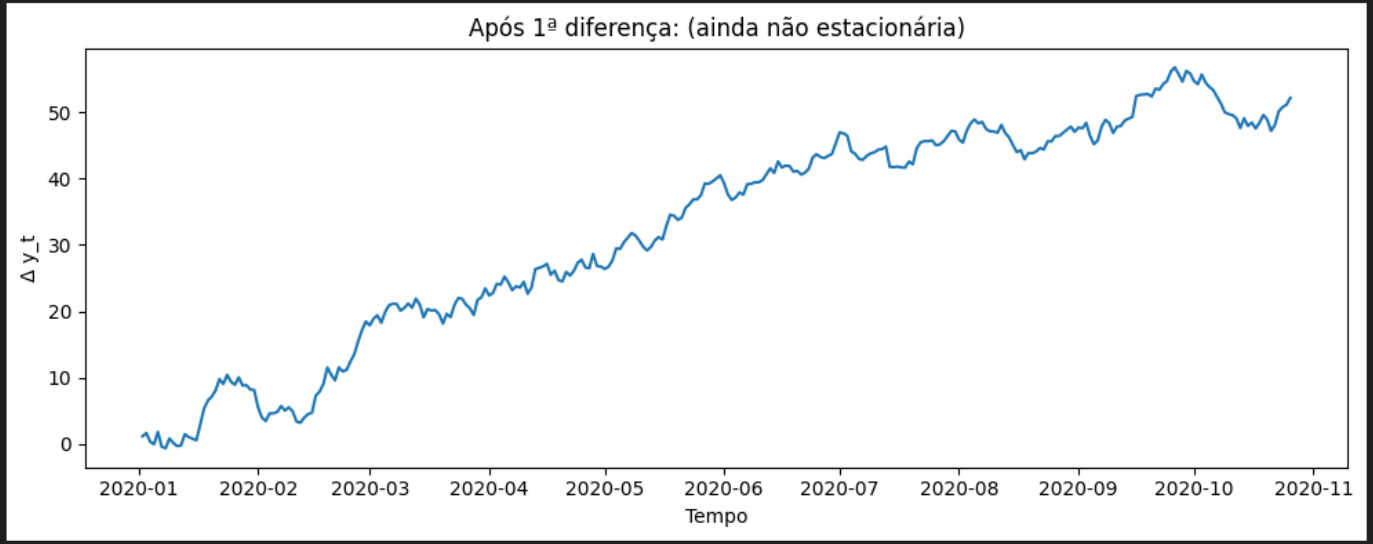


Testando e diferenciando 1 vez:

```
# teste antes
adf_orig = teste_adf(serie)
kpss_orig_c = teste_kpss_(serie)

# 1 diferenciacao
diff1 = serie.diff().dropna()


plt.figure(figsize=(10,4))
plt.plot(diff1.index, diff1.values)
plt.title("Após 1ª diferença: (ainda não estacionária)")
plt.xlabel("Tempo")
plt.ylabel("Δ y_t")
plt.tight_layout()
plt.show()

# teste depois da 1 diferenciacao
adf_d1 = teste_adf(diff1)
kpss_d1_c = teste_kpss_(diff1)
```

Testando e diferenciando 1 vez:

```
ADF
  Estatística ADF: 0.32725473655772774
  p-valor: 0.9785609389712293
  Críticos 1/5/10%: {'1%': -3.4524859843440754, '5%': -2.871288184343229, '10%': -2.571964047565425}

KPSS
  Estatística KPSS: 2.745591144854625
  p-valor: 0.01
  Críticos 10/5/2.5/1%: {'10%': 0.347, '5%': 0.463, '2.5%': 0.574, '1%': 0.739}
```




```
ADF
  Estatística ADF: -1.5342670715280329
  p-valor: 0.5164969774334552
  Críticos 1/5/10%: {'1%': -3.4524859843440754, '5%': -2.871288184343229, '10%': -2.571964047565425}

KPSS
  Estatística KPSS: 2.6827548505987515
  p-valor: 0.01
  Críticos 10/5/2.5/1%: {'10%': 0.347, '5%': 0.463, '2.5%': 0.574, '1%': 0.739}
```

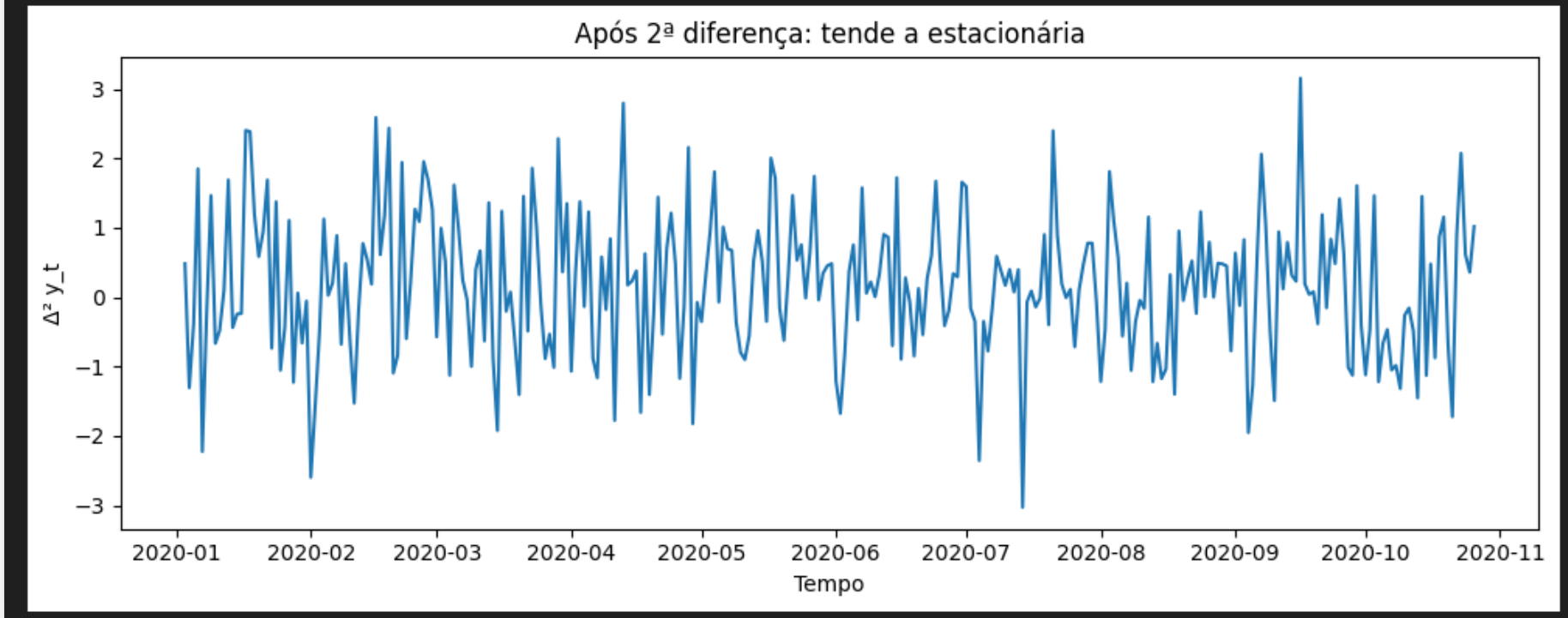
Testando e diferenciando 2 vez:

Testando estacionariedade após 2º diferenciacao

```
diff2 = diff1.diff().dropna()
adf_d2 = teste_adf(diff2)
kpss_d2_c = teste_kpss_(diff2)

plt.figure(figsize=(10,4))
plt.plot(diff2.index, diff2.values)
plt.title("Após 2ª diferença: tende a estacionária")
plt.xlabel("Tempo")
plt.ylabel("Δ² y_t")
plt.tight_layout()
plt.show()
```

Testando e diferenciando 2 vez:

```
ADF
  Estatística ADF: -16.38521726937397
  p-valor: 2.727219902485694e-29
  Críticos 1/5/10%: {'1%': -3.4525611751768914, '5%': -2.87132117782556, '10%': -2.5719816428028888}

KPSS
  Estatística KPSS: 0.15803308861076282
  p-valor: 0.1
  Críticos 10/5/2.5/1%: {'10%': 0.347, '5%': 0.463, '2.5%': 0.574, '1%': 0.739}
```



Diferenciação Sazonal

Diferenciação Sazonal:

Se a série tem **padrão repetitivo (mensal, semanal etc.)**, usamos a diferença sazonal

$$\nabla_s y_t = y_t - y_{t-s}$$

Onde ***s* é o período da sazonalidade**. Exemplo: s = 12 para dados mensais com sazonalidade anual

Implementação: gerando

- Vamos gerar uma série não estacionária e visualiza-la para aplicarmos a diferenciação

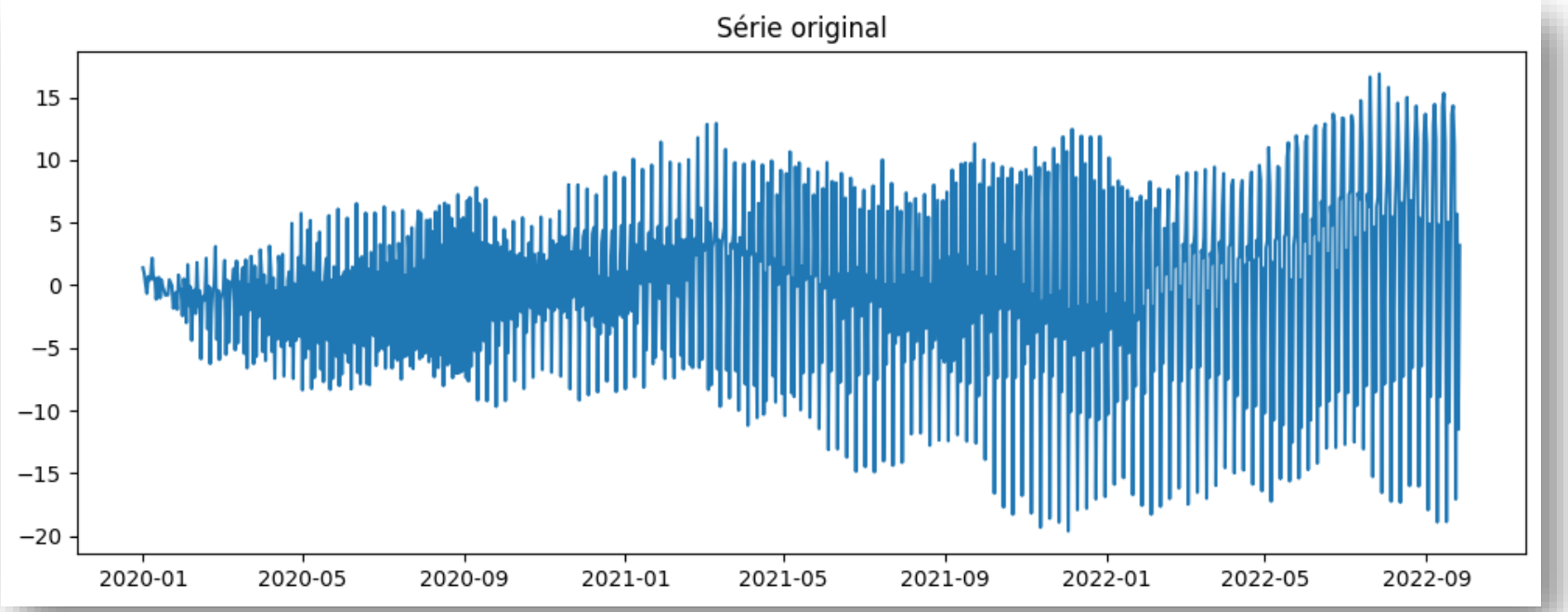
```
np.random.seed(42)
n = 1000
s = 7
eps = np.random.normal(0, 1, n)
y = np.zeros(n)
y[:s] = np.random.normal(0, 1, s)
for t in range(s, n):
    y[t] = y[t - s] + eps[t]

idx = pd.date_range("2020-01-01", periods=n, freq="D")
serie = pd.Series(y, index=idx, name="y")
```

Testando

Realize a plotagem da serie e teste sua estacionariedade com ADF e KPSS

Testando

Realize a plotagem da serie e teste sua estacionariedade com ADF e KPSS



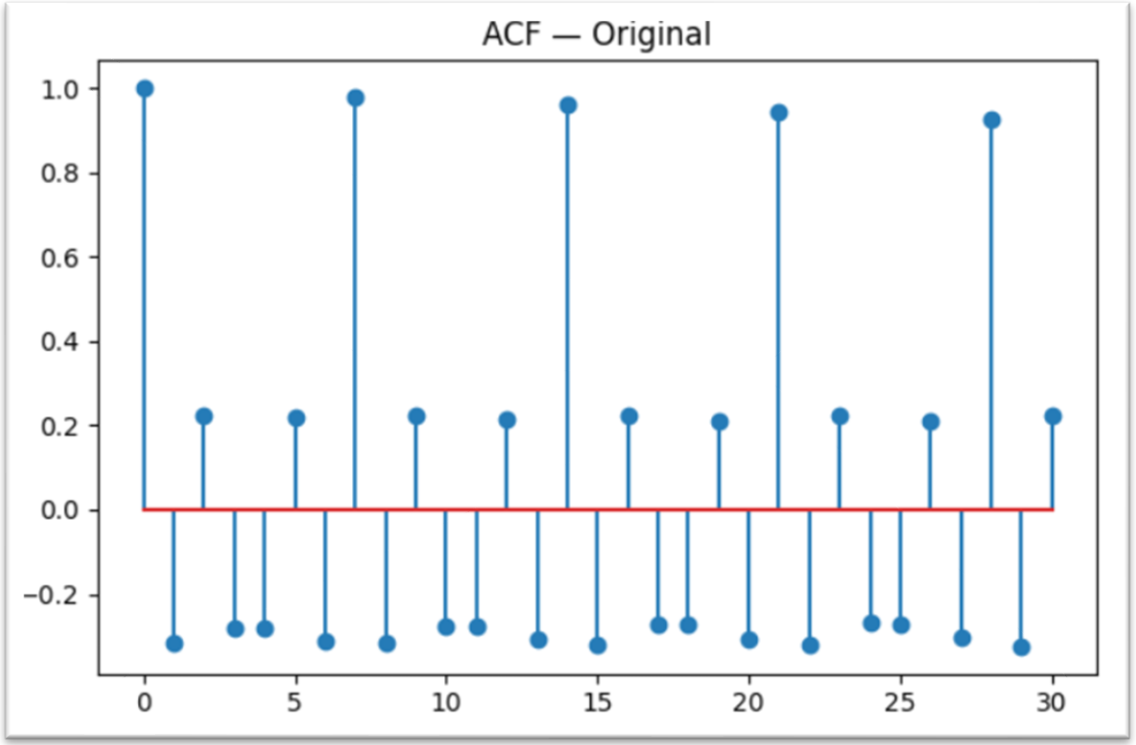

```
ADF p-valor: 0.766688
KPSS (nível): estat=0.4065  p≈0.074
```

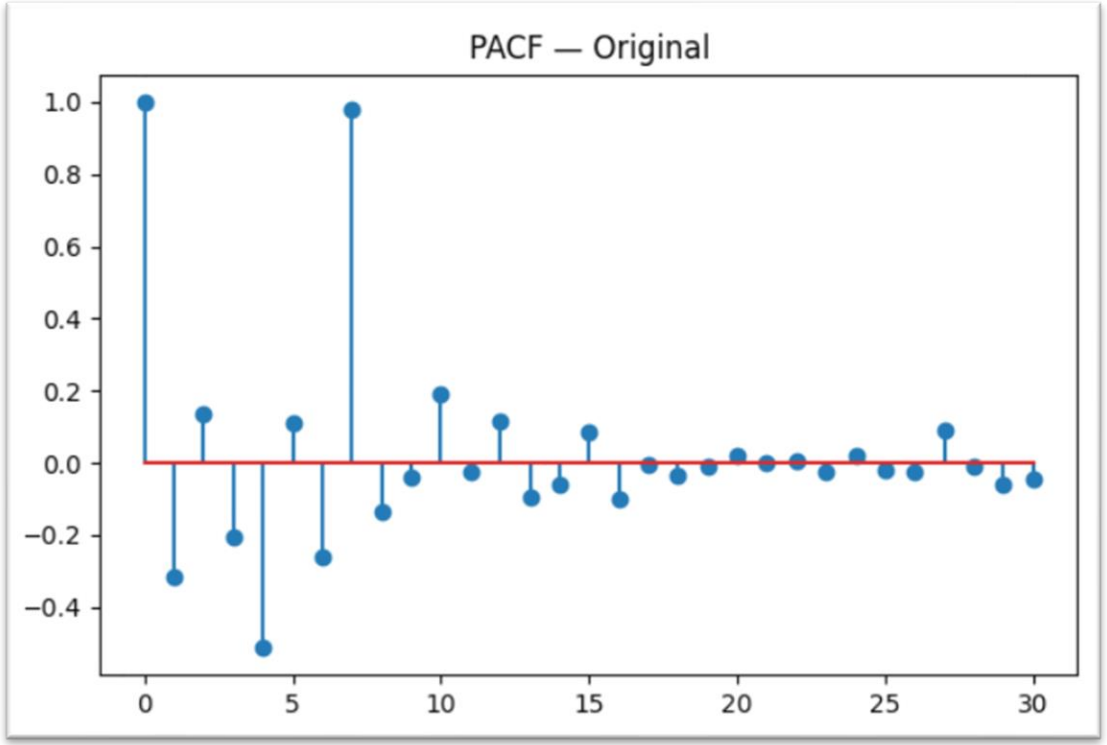
Testando: ACF e PACF

Realize os plots ACF e PACF da série original

Testando: ACF e PACF

Realize os plots ACF e PACF da série original



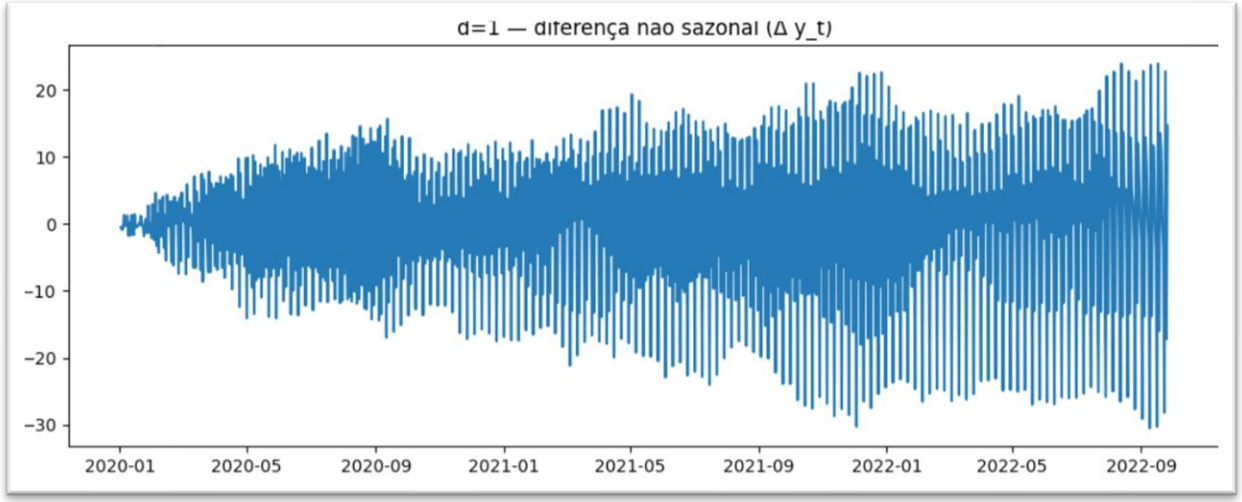


Diferenciação: ordem 1

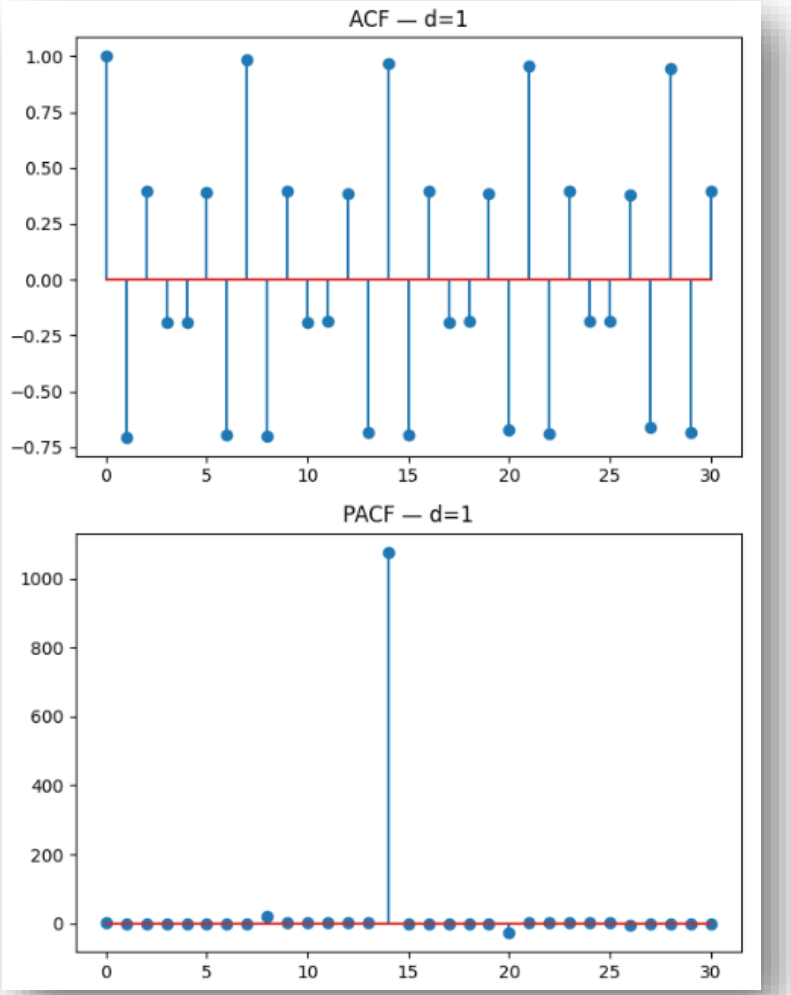
Diferencie em ordem 1, plote a serie, faça os testes e plote ACF e PACF

Diferenciação: ordem 1

Diferencie em ordem 1, plote a serie, faça os testes e plote ACF e PACF

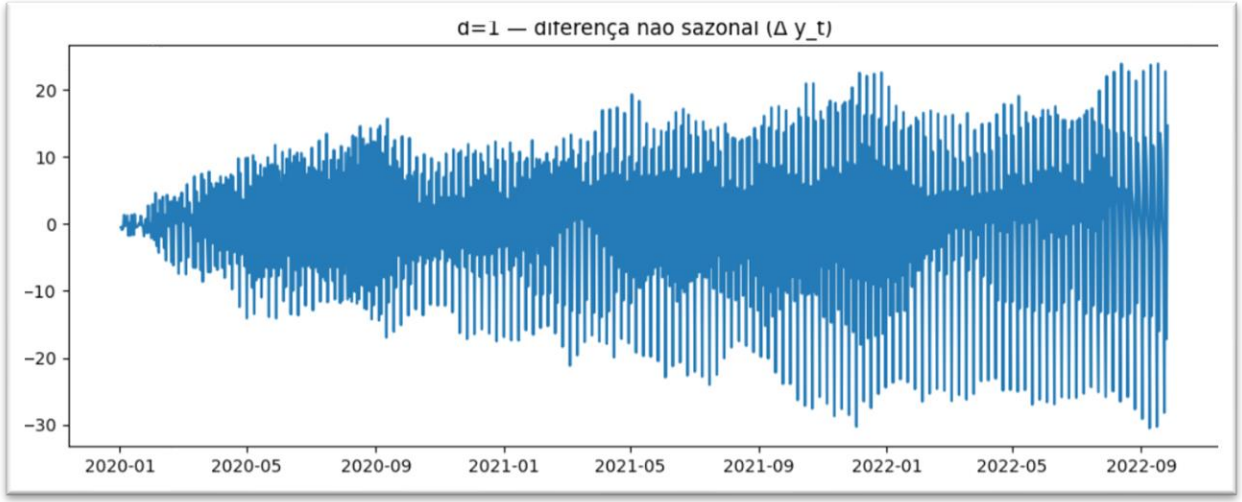



```
ADF p-valor: 0.000000
KPSS (nível): estat=0.0174  p≈0.100
```



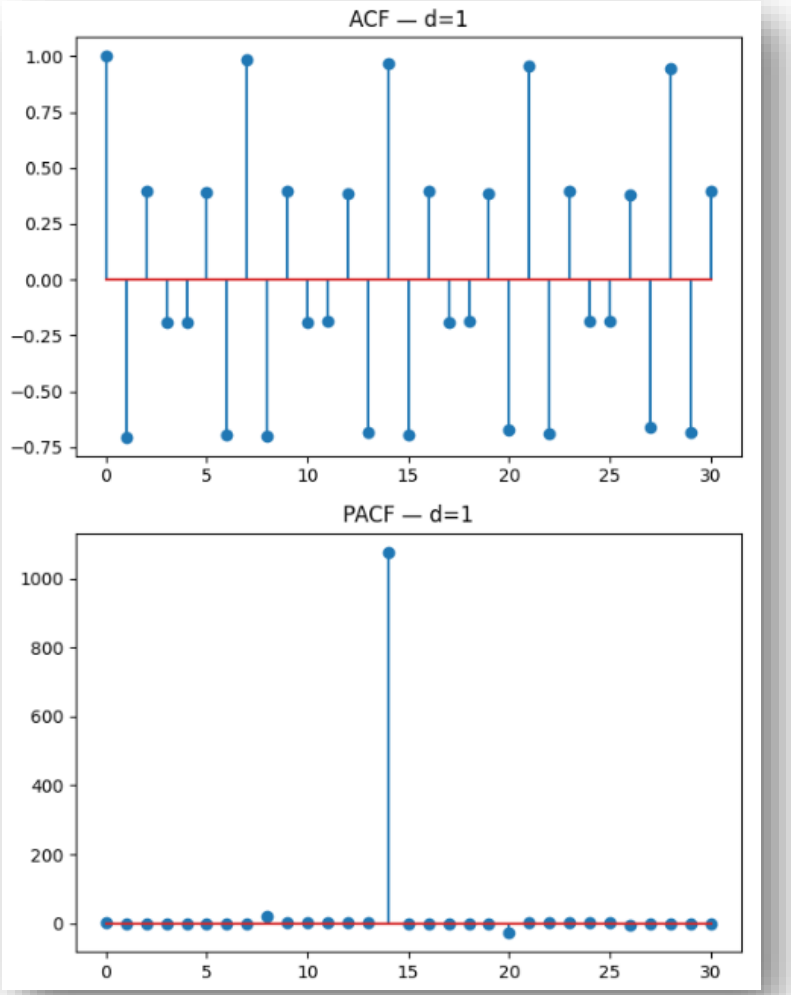
Diferenciação: ordem 1

Diferencie em ordem 1, plote a serie, faça os testes e plote ACF e PACF



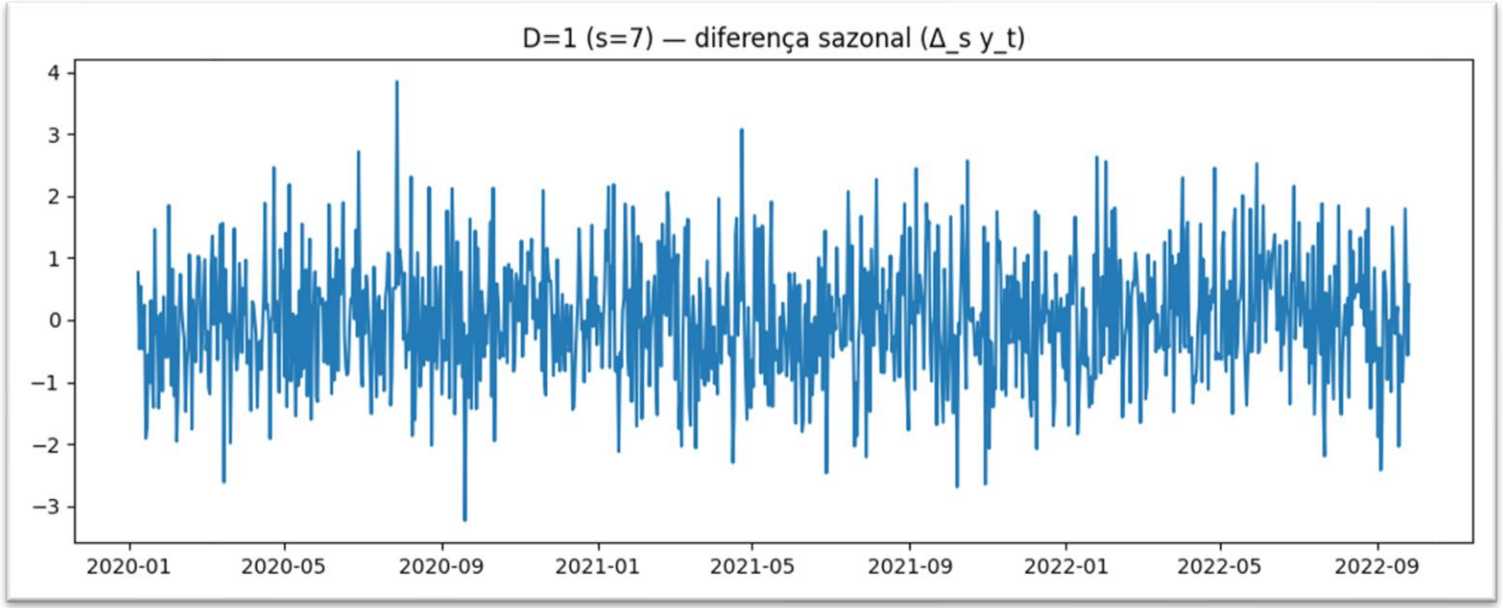

```
ADF p-valor: 0.000000
KPSS (nível): estat=0.0174  p≈0.100
```

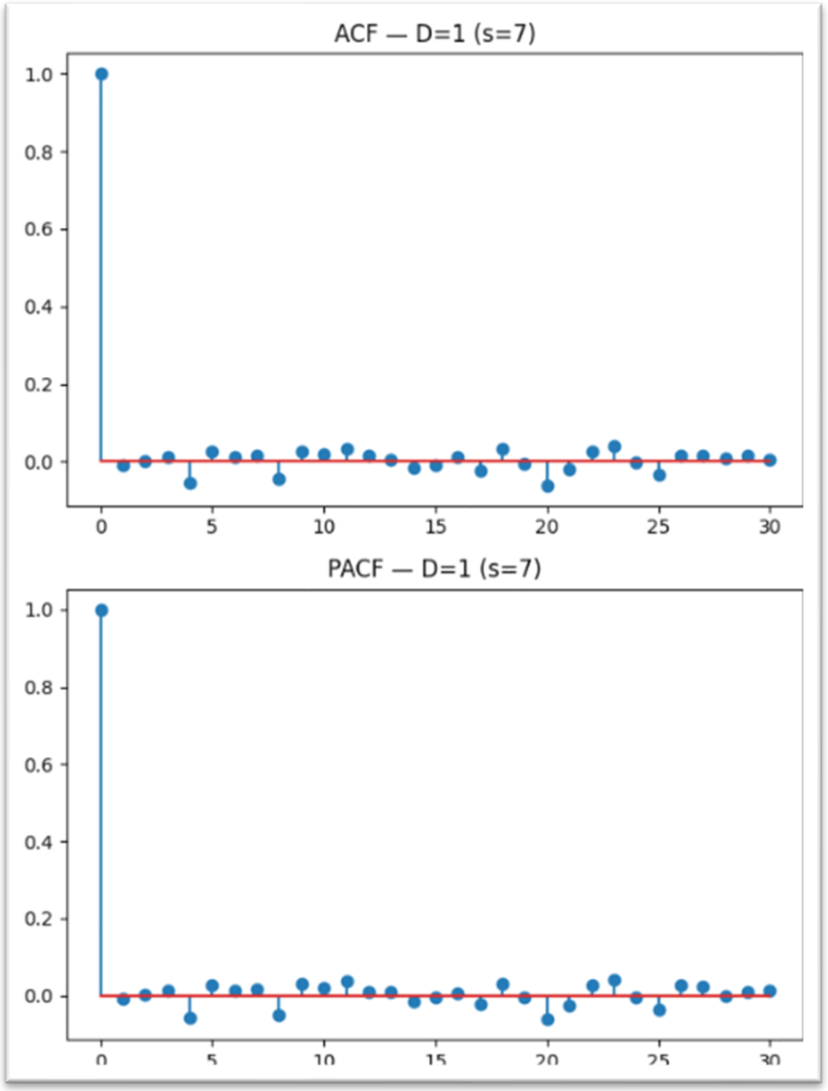
- **ADF/KPSS garantem estacionariedade não-sazonal.**
- **Para estacionariedade sazonal, sempre confirme pela ACF/PACF**
- **No R temos HEGY para testar raízes unitárias sazonais**



Diferenciação sazonal: ordem 1, s=7

Diferencie a serie original em ordem 1 com s=7. Apresente todos os plots e testes:






```
diff_s = serie.diff(s).dropna()
```

```
ADF p-valor: 0.000000
KPSS (nível): estat=0.2305  p≈0.100
```

Diferenciação Resumo:

Tipo de diferenciação	Uso principal
Primeira diferença	Remove tendência linear
Segunda diferença	Remove tendência quadrática
Diferença de ordem d	Remove tendências mais complexas
Diferença sazonal	Remove sazonalidade
Combinação	Remove tendência e sazonalidade ao mesmo tempo

Desafio

Desafio:

Recupere as séries não estacionárias vistas até aqui no curso e realize as diferenciações para tentar transforma-las em estacionárias

Instituto J&F

Centro de Educação para os Negócios